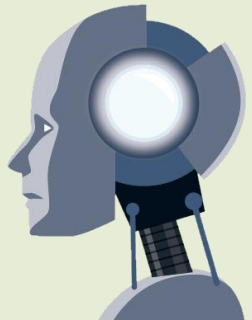
02

정보의 표현

단원 목차

2.1 컴퓨터의 자료 표현
2.2 진수와 수의 표현
2.3 컴퓨터의 정보 종류
2.4 문자와 논리 표현
2.5 컴퓨터의 연산



사람에게 친숙한 10진수

수에서 하나의 자릿수(digits)에 사용하는 숫자가 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 까지 열 개이므로 10진수

- **10이라는 것을 기수(base)**
  - 10진수에서 가장 오른쪽은 10^0인 단 단위
  - 그 왼쪽 옆은 10^1인 십 단위, 다시 그 왼쪽 옆은 10^2인 백 단위의 자릿수
  - 어느 자릿수는 바로 근접한 오른쪽 자릿수의 10배로 커진 자릿수

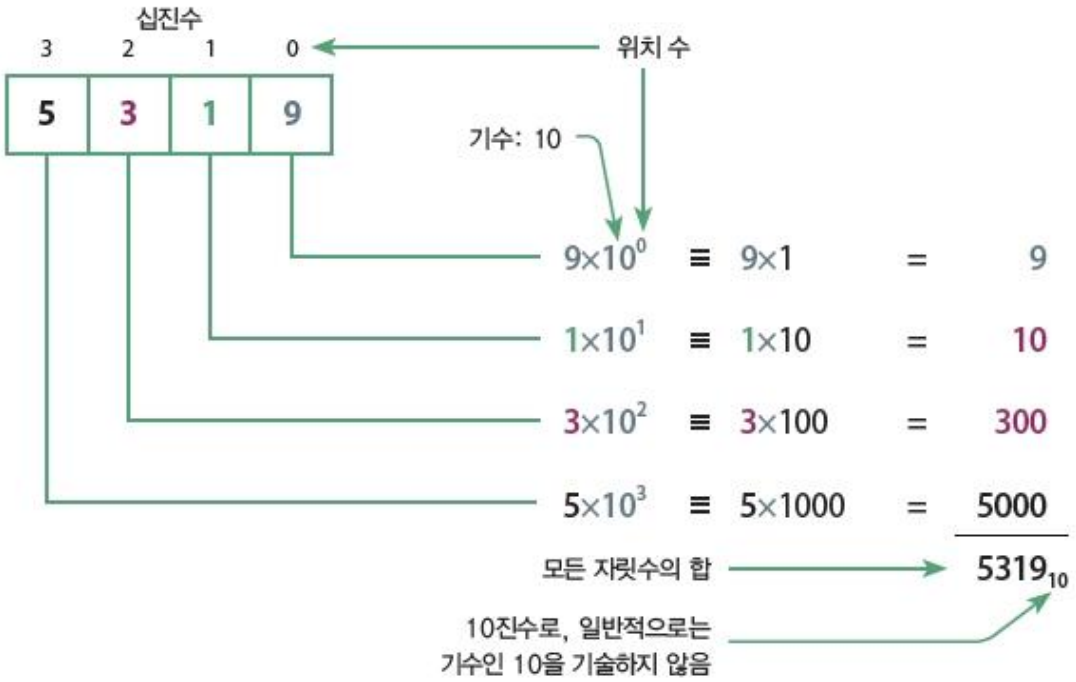


그림 3.1 ▸ 10진수 5319의 표현과 이해

컴퓨터 내부 자료 표현

컴퓨터 내부에는 전기가 흐르거나(On) 흐르지 않는(Off) 두 가지 전기 신호만을 표현 가능

- **두 가지 정보 표현**
  - 컴퓨터 내부에서는 전기가 흐르거나(On) 흐르지 않는(Off) 두 가지 전기 신호만을 표현
    - **트랜지스터를 이용하여 자료를 처리하고 저장**
  - 2진수 체계를 사용
    - **전기가 흐를 경우 '참'을 의미하는 '1' , 흐르지 않을 경우 '거짓'의 '0' 으로 표현**
- **컴퓨터 내부에서 2진수 체계를 사용**
  - 숫자는 0과 1을 표현
  - 컴퓨터는 논리의 조합이 간단하고 내부에 사용되는 소자의 특성상 이진법을 사용하는 것이 가장 합리적이고 효율적인 방식

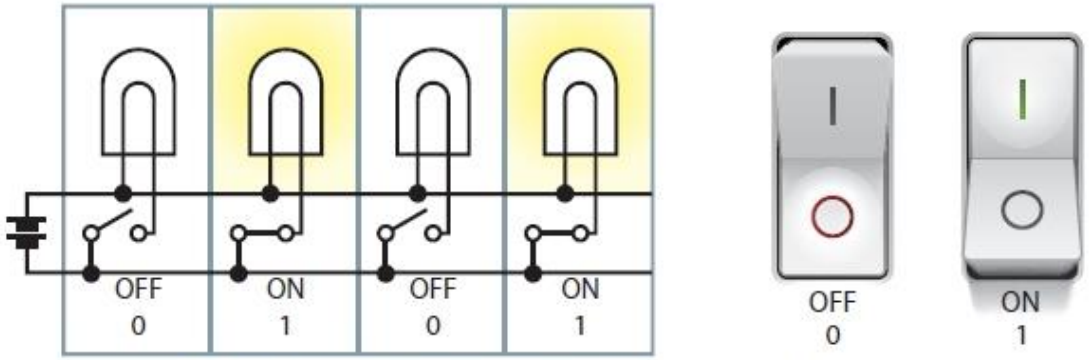


그림 3.2 ▸ 트랜지스터와 스위치에서의 0과 1의 이진 표현

2진수의 이해

2진수(binary number)는 11010, 1010, 110과 같이 사용할 수 있는 숫자가 0과 1, 2 개이므로 2진수

- **10진수의 기수(base, radix)가 10**
  - 2진수의 기수는 2

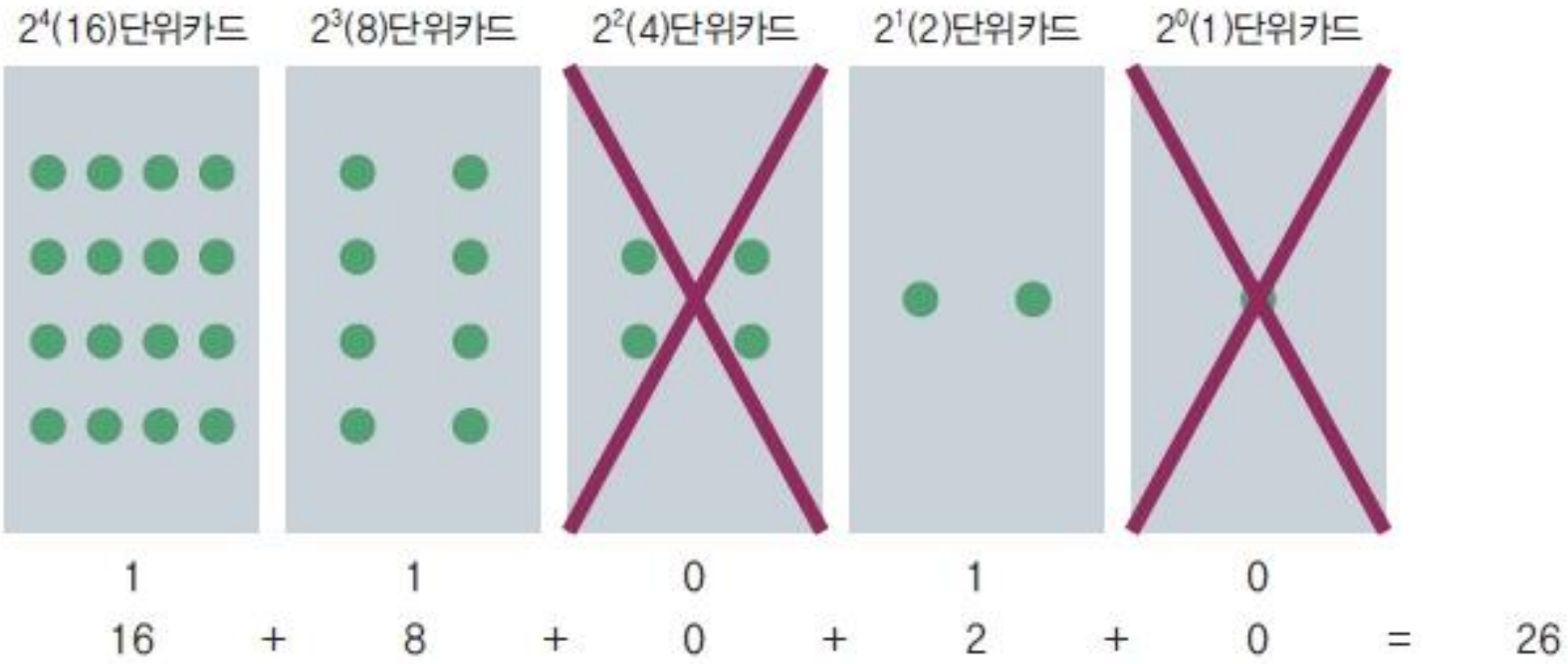


그림 3.3 ▸ 카드를 이용한 2진수의 이해

비트와 바이트

비트(bit)는 BInary digiT의 합성어로 컴퓨터 메모리의 저장 단위 또는 정보 처리 단위 중에서 가장 작은 단위를 말함

- **두 가지 정보 표현**
  - 전기의 흐름 상태인 온(on)과 오프(off)를 표현하는 단위가 비트
    - **1과 0인 2진수로 표현이 가능**
- **비트(Bit)**
  - 컴퓨터의 정보 처리 단위 중에서 가장 작은 정보 단위
- **바이트(byte)**
  - 비트가 연속적으로 8개 모인 정보 단위

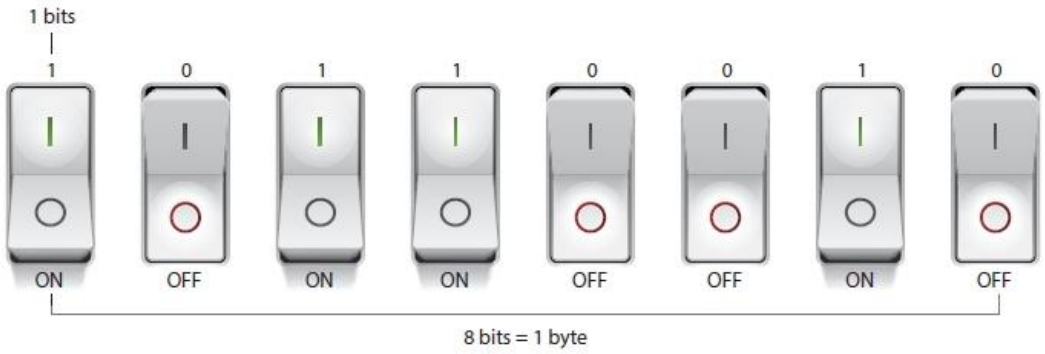


그림 3.4 ▸ 트랜지스터의 전기적 스위치로 구성된 8비트 10110010의 표현

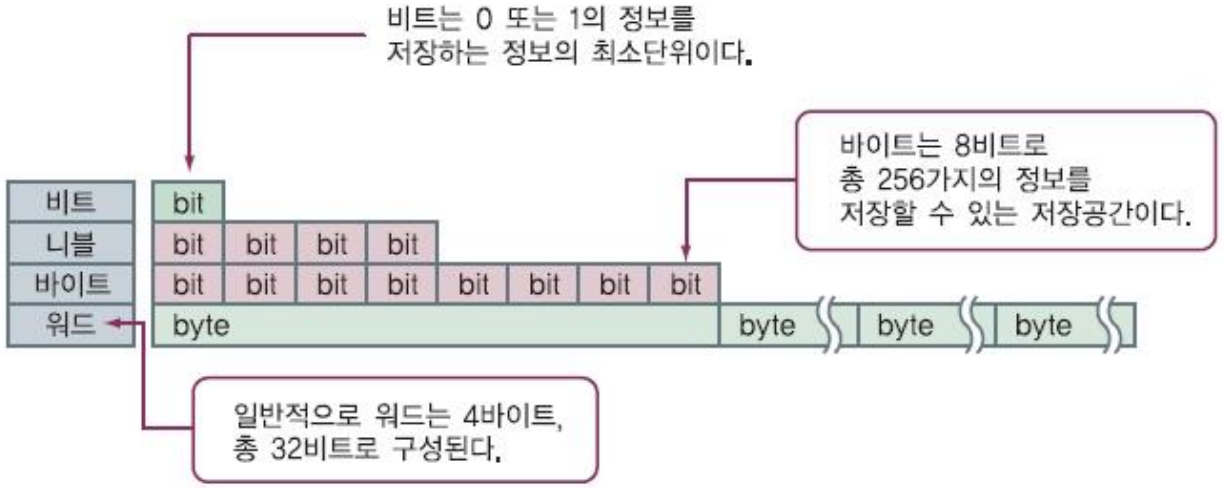


그림 3.5 ▸ 비트와 니블, 바이트, 워드

저장 용량

테라 바이트(Tera Byte)는 2^{40}, 페타 바이트(Peta Byte)는 2^{50}, 엑사 바이트(Exa Byte)는 2^{60}

- **바이트가 정보의 용량 단위**
  - 킬로, 메가, 기가, 테라
  - 페타, 엑사, 제타, 요타

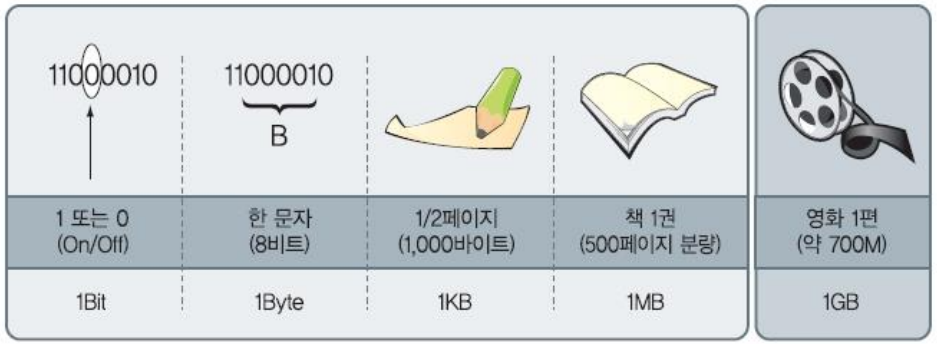


그림 3.6 ▸ 저장 용량의 비교

표 3.1 저장 용량 단위

표기	단위	계산	바이트 수	계량 단위
B	Byte	2^{0}	1	
KB	Kilo Byte	2^{10}	1,024	천
MB	Mega Byte	2^{20}	1,048,576	백만
GB	Giga Byte	2^{30}	1,073,741,824	십억
TB	Tera Byte	2^{40}	1,099,511,627,776	조
PB	Peta Byte	2^{50}	1,125,899,906,842,624	천조
EB	Exa Byte	2^{60}	1,152,921,504,606,846,976	백경
ZB	Zetta Byte	2^{70}	1,180,591,620,717,411,303,424	십해
YB	Yotta Byte	2^{80}	1,208,925,819,614,629,174,706,176	자

진수의 종류

10진수의 각 자리는 오른쪽부터 1(10^0)자리, 10(10^1)자리, 100(10^2)자리 순으로 그 자리에 따라 의미하는 수가 정해짐

- **10진수**
  - 우리 인간이 일상 생활에서 이용하는 가장 친숙한 진수
- **N진수**
  - 0에서 N-1까지의 정수를 이용
  - 오른쪽부터 n번째 자리의 크기는 N^{n-1}

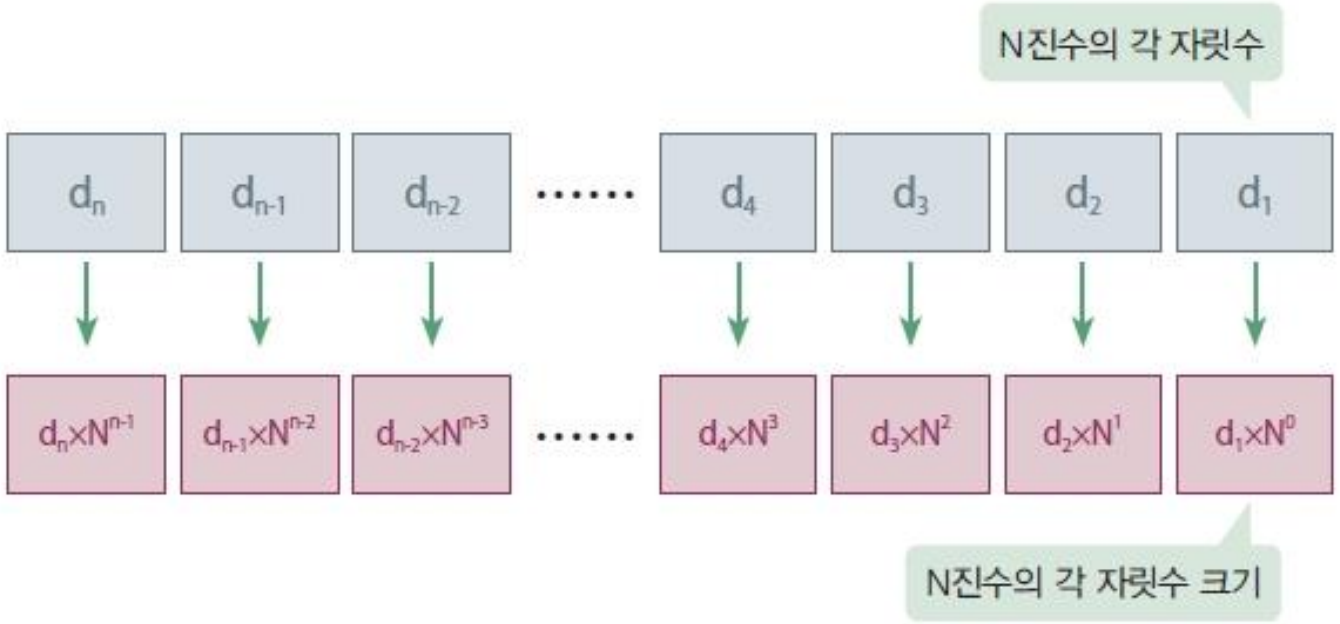


그림 3.8 ▸ N진수에서의 자릿수와 그 크기

2진수

2진수는 0과 1의 두 가지 표현으로 각 자릿수를 표시하는 진수

- **2진수**
  - 0과 1의 두 가지 표현으로 각 자릿수를 표시
  - 라이프니츠(Gottfried Wilhelm von Leibniz 1646~1716)
    - **독일의 철학자, 법학자이면서 수학자**
    - **미적분과 함께 현대 대표 문명 기기인 컴퓨터가 탄생하는 데 결정적인 '0과 1'의 이분법을 개발**

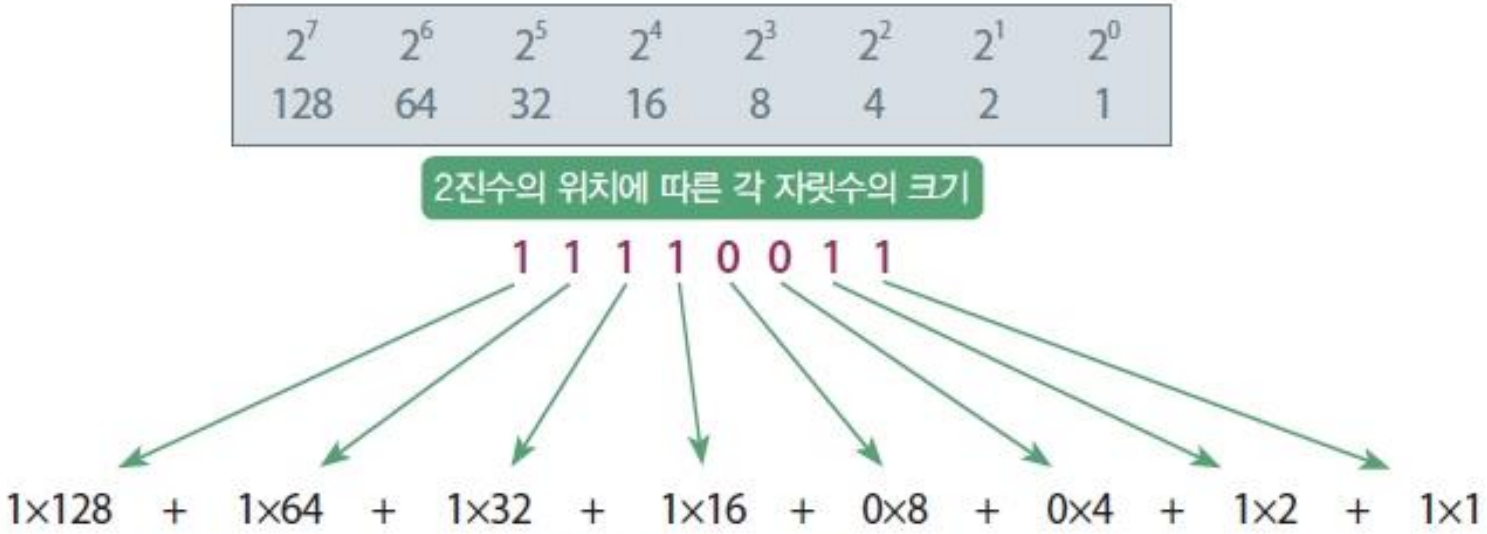


그림 3.9 ▸ 2진수의 의미

8진수와 16진수

2진수의 표현은 쉽게 8진수와 16진수로 변환이 가능한 진수

- **8진수**
  - 0부터 7까지의 8가지의 수를 이용하여 숫자를 표시

$$
\begin{aligned}
301_8 &= 3\times 8^2 + 0\times 8^1 + 1\times 8^0 \\
&= 192 + 0 + 1 \\
&= 193
\end{aligned}
$$

- **16진수**
  - 총 16개의 숫자나 문자를 사용하여 표시
    - **0에서 9까지 그리고 A에서 F까지**
      - **소문자 a에서 f까지도 가능**

$$
\begin{aligned}
1AF_{16} &= 1\times 16^2 + A\times 16^1 + F\times 16^0 \\
&= 256 + 160 + 15 \\
&= 431
\end{aligned}
$$

10진수를 2진수로 변환

전통적인 방법, 2로 나눈 나머지를 역순으로 나열

- **$26 = 11010_2$**

단계 1 : 주어진 값을 2로 나누고 그 나머지를 기록한다.
단계 2 : 몫이 0이 아니면 계속해서 새로운 몫을 2로 나누고
그 나머지는 기록한다.
단계 3 : 몫이 0이면 원래 값의 2진 표현은 나머지가 기록되는 순서대로
왼쪽에서 오른쪽으로 나열한다.

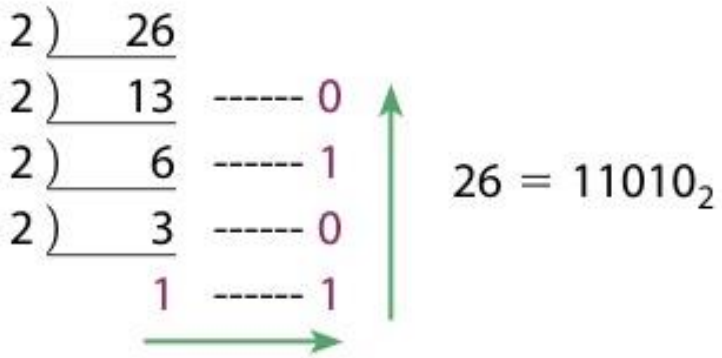


그림 3.10 ▸ 10진수 26을 2진수로 변환하는 과정

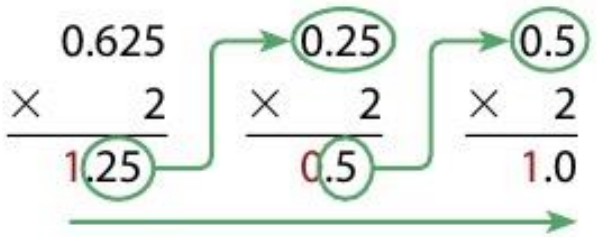
10진수의 소수를 2진수로 변환

2를 곱해 나온 단 단위를 소수부분이 0이 될 때까지 나열

- **$0.625 = 0.101_2$**

단계 1 : 10진수에 2를 곱하여 나온 결과에서 정수 부분으로의 자리 올림수와 소수점 아래 부분을 따로 보관한다.
단계 2 : 단계 1에서 소수 부분이 0이면 단계 3으로 넘어가고, 아니면 소수점 아래 부분을 다시 새로운 10진수로 하여 단계 1을 반복한다.
단계 3 : 구해진 정수 부분으로의 자리 올림수를 순서대로 나열한다.

- $0.5 = ½ = 0.1_2$
- $0.25 = ¼ = 0.01_2$
- $0.125 = ⅛ = 0.001_2$
- $0.0625 = 1/16 = 0.0001_2$
- $0.75 = 0.11_2$
- $0.375 = 0.011_2$
- $0.6875 = 0.1011_2$



$0.625 = 0.101_2$

그림 3.11 ▸ 10진수 소수 0.625의 2진수 변환과정

2진수, 8진수, 16진수간 상호관계

수식 $8 = 2^3$, $16 = 2^4$ 이 만족하므로 2진수로 표현된 수를 각각 8진수, 16진수로 표현하는데 쉽게 변환이 가능

- **변환이 간편**
  - 소수점을 기준으로 정수 부분은 왼쪽으로,
  - 소수 부분은 오른쪽으로
  - 2진수 → 16진수
    - **2진수의 4자리 씩을 16진수로 변환**
  - 2진수 → 8진수
    - **마찬가지 방법으로 2 진수의 3자리 씩을 8진수로 변환**

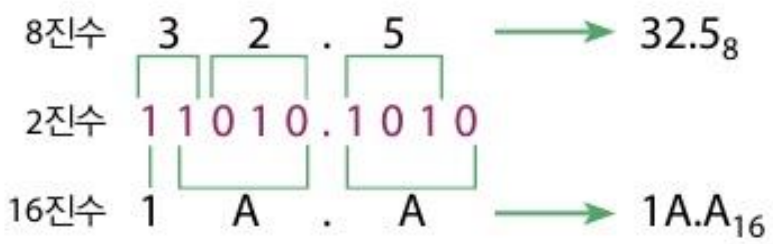


그림 3.12 ▸ 2진수의 표현을 8진수와 16진수로 변환하는 과정

2진수의 음수 표현: 1의 보수

2진수의 음수를 표현하는 가장 일반적인 방법은 2의 보수 표기 방법

- **1의 보수(1's complement)**
  - 주어진 이진수의 비트를 0은 1로, 1은 0으로 각각 변환하는 방법
  - 이진수 0100의 1의 보수는 1011
  - 0이 +0과 -0으로 각각 다르다는 단점
    - **+0은 0000이고 -0은 1111**

그림 3.13 ▸ 2진수를 1의 보수로 변환하는 과정

2진수의 음수 표현: 2의 보수

음수의 2진수 표기인 2의 보수를 구하는 방법 3가지

- **2의 보수 변환 방법**
  - 방법 1

n비트에서 –a의 2의 보수 계산 방법 : $2^n - a$

n비트에서 –a의 2의 보수 계산 방법:

$$2^n - a$$

- **-4를 4비트로 2의 보수로 표현**
  - **표현되는 비트의 수가 4개이므로 $2^4 - 4 = 12 = 1100_2$**
  - 방법 2

단계 1 : 음수의 절대값인 양의 정수의 이진수를 n 비트에서 구한다.
단계 2 : 단계 1에서 얻은 이진수의 1의 보수를 n 비트에서 구한다 .
단계 3 : 단계 2에서 얻은 이진수에 1을 더한 n 비트 만을 취한다.

표 3.3 4비트에서 –4의 2의 보수 계산 과정

단계	방법	결과
1단계(양수를 2진수로)	양수 2를 4비트의 2진수로	0100
2단계(1의 보수 구하기)	4비트에서 비트를 각각 0은 1로, 1은 0으로	1011
3단계(1 더하기)	4비트에서 1 더하기	1100

2진수의 음수 표현: 2의 보수

양수의 n비트 2진수에서 가장 오른쪽의 0에서 처음으로 나오는 1까지 그대로 두고 나머지 왼쪽 비트를 모두 1의 보수로 바꾸는 방법

- **방법 3**
  - 가장 간편
    - **-4를 4비트로 2의 보수로 표현하자면**
    - **가장 오른쪽의 0에서 처음으로 나오는 1까지인 100은 그대로 두고**
    - **나머지 비트인 0만을 1의 보수인 1로 바꾸면 바로 1100의 결과**

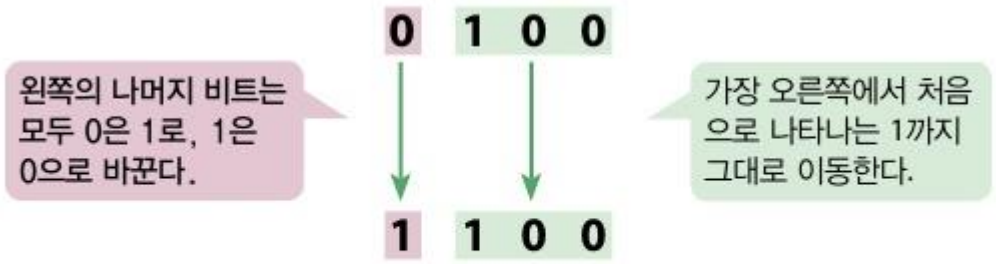


그림 3.15 ▸ 2의 보수 계산 방법 3(간편한 방법)

4비트의 2진수의 표현

2의 보수는 1의 보수의 단점을 보완하고 4비트에서 -8에서 +7까지 표현이 가능

- **1의 보수 단점**
  - 1의 보수는 0이 두 가지로 표현되므로 -8을 표현 불가능
- **2의 보수**
  - 2의 보수는 1의 보수의 단점을 보완
  - 4비트에서 -8에서 +7까지 표현이 가능

표 3.4 4비트 2진수의 1의 보수와 2의 보수

숫자	2진수(양수)	1의 보수	2의 보수
0	0000	0000 1111	0000
−1	0001	1110	1111
−2	0010	1101	1110
−3	0011	1100	1101
−4	0100	1011	1100
−5	0101	1010	1011
−6	0110	1001	1010
−7	0111	1000	1001
−8	1000	–	1000

2진수의 표현: 최상위 비트

수의 비트 표현에서 가장 왼쪽의 비트를 최상위 비트(MSB: Most Significant Bit)

- **부호 비트(sign bit)**
  - 보수 표현에서는 이 최상위비트가 부호를 나타내므로 부호비트(sign bit)라고도 부름
  - 2의 보수 모두 음수는 4비트의 가장 왼쪽 비트가 1

그림 3.16 ▸ MSB와 1의 보수와 2의 보수의 표현 범위

부호가 있는 정수 표현

컴퓨터는 정수의 양수와 음수를 표현하는데 주로 2진수와 2의 보수 방법을 이용

- **8비트의 메모리**
  - 2^8가지(256)의 정보를 표현 가능
- **n개의 비트로 정수의 양수와 음수를 모두 표현**
  - 수의 범위
    - **$-(2^{n-1})$ ~ $+(2^{n-1}-1)$**

표 3.5 8비트로 구성되는 정수의 표현

숫자	2진수	숫자	2진수
–	–	−128	10000000
+127	01111111	−127	10000001
+126	01111110	−126	10000010
…	…	…	
+5	00000101	−5	11111011
+4	00000100	−4	11111100
+3	00000011	−3	11111101
+2	00000010	−2	11111110
+1	00000001	−1	11111111
0	00000000	–	–

표 3.6 n비트로 구성되는 정수의 표현 범위

저장공간 크기	표현 범위
1비트	$-1(-2^0) \sim 0(2^0-1)$
2비트	$-2(-2^1) \sim 1(2^1-1)$
4비트	$-8(-2^3) \sim 7(2^3-1)$
8비트	$-128(-2^7) \sim 127(2^7-1)$
16비트	$-32,768(-2^{15}) \sim 32,767(2^{15}-1)$
32비트	$-2,147,483,648(-2^{31}) \sim 2,147,483,647(2^{31}-1)$

부호가 없는 정수 표현

0과 양수만을 다루는 정수를 부호가 없는(unsigned) 정수

- **unsigned 정수**
  - 2^8가지(256)의 정보를 표현 가능
- **오버플로우**
  - n비트의 메모리에 표현 범위를 초과하는 수의 값을 저장하는 경우

표 3.7 4비트 2진수의 singed 정수와 unsigned 정수

4비트 정보	unsigned	signed 1의 보수	signed 2의 보수	4비트 정보	unsigned	signed 1의 보수	signed 2의 보수
0000	0	0	0	1000	8	−7	−8
0001	1	1	1	1001	9	−6	−7
0010	2	2	2	1010	10	−5	−6
0011	3	3	3	1011	11	−4	−5
0100	4	4	4	1100	12	−3	−4
0101	5	5	5	1101	13	−2	−3
0110	6	6	6	1110	14	−1	−2
0111	7	7	7	1111	15	−0	−1

$M + N = 0011_2 + 0110_2 = 1001_2$인데, 이 수 1001_2는 2의 보수 방법으로 −7이라는 결과가 나온다.

부동소수점 수와 정규화

실수를 표현하는 방식을 부동소수점 수(floating point number)

- **부동소수점 수(floating point number)**
  - 실수 표현 방식
- **정규화(normalization)**
  - 10진수의 실수 352.45
    - **3.5245×10^2으로도 표현이 가능**
  - 정규화(Normalization) 방법
    - **실수의 소수점을 이동**
    - **소수점 왼쪽에 단 하나의 자릿수가 오도록 하고**
    - **소수점의 원래 위치는 진수의 지수로 표현하는 방법**
  - 정규화된 실수 표현을 부동소수(Floating Point Number)
    - **부동 소수는 수의 소수점의 위치를 움직일 수 있게 한다는 의미**

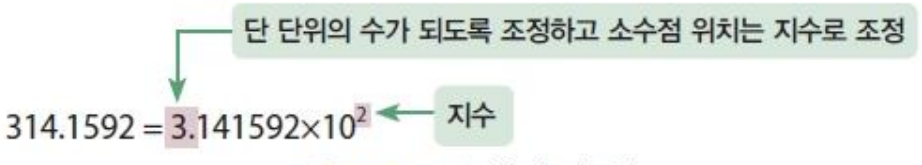


그림 3.17 ▸ 10진수의 정규화

표 3.8 10진수의 정규화의 예

10진수 실수	정규화
1234.5432	1.2345432×10^3
24.345078	2.4345078×10^1
0.003045	3.045×10^{-3}
−134.784556	-1.34784556×10^2

정규화에서 지수와 가수

26.625를 이진수로 표현하면 11010.101, 부동소수로 표현하면 부호, 지수와 가수로 구분하여 표시

- **부동소수**
  - 소수 부분과 지수 부분으로 구분
    - **이를 각각 가수(Mantissa)와 지수(Exponent)**
    - **가수**
      - **수의 정밀도(Precision)**
    - **지수**
      - **수의 크기(Magnitude) 표현**

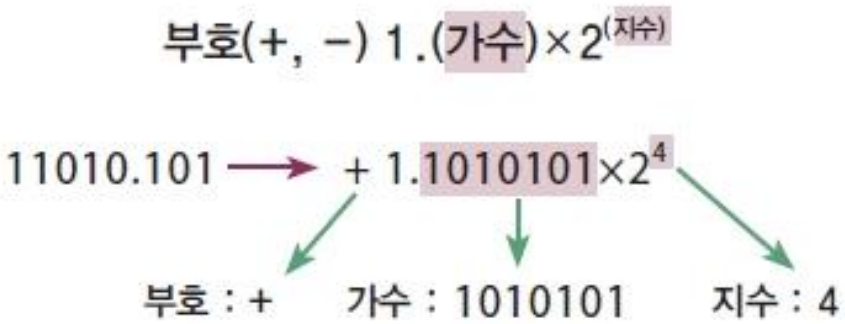


그림 3.18 ▸ 2진수 11010.101의 정규화 표현

- **부동소수의 표현**
  - 부호부(Sign), 지수부(Exponent), 가수부(Mantissa)로 나누어 이진 표현으로 저장

표 3.9 2진수 실수를 정규화된 부동소수의 부호, 지수, 가수로 표현

2진수 실수	정규화	부호	지수	가수
101.11	1.0111×2^{2}	0(+)	2	0111
10011.101	1.0011101×2^{4}	0(+)	4	0011101
0.001011	1.011×2^{-3}	0(+)	-3	011
-0.00000111	-1.11×2^{-6}	1(-)	-6	11

부동소수의 저장 표현

전기전자기술자협회(IEEE)에서는 부동소수를 저장하는 표준을 제공

- **2가지 방식**
  - 단정도 형식(single precision format): 32비트
  - 배정도 형식(double precision format): 64비트
- **부호, 지수, 가수**
  - 부호부: 양수이면 0, 음수이면 1이 저장
  - 지수부:
    - **단정도에서는 127 초과 이진수**
    - **배정도에서는 1023 초과 이진수로 지수를 저장**
  - 가수부:
    - **부동소수의 가수를 왼쪽부터 저장**
    - **오른쪽 나머지 비트는 모두 0으로 채움**

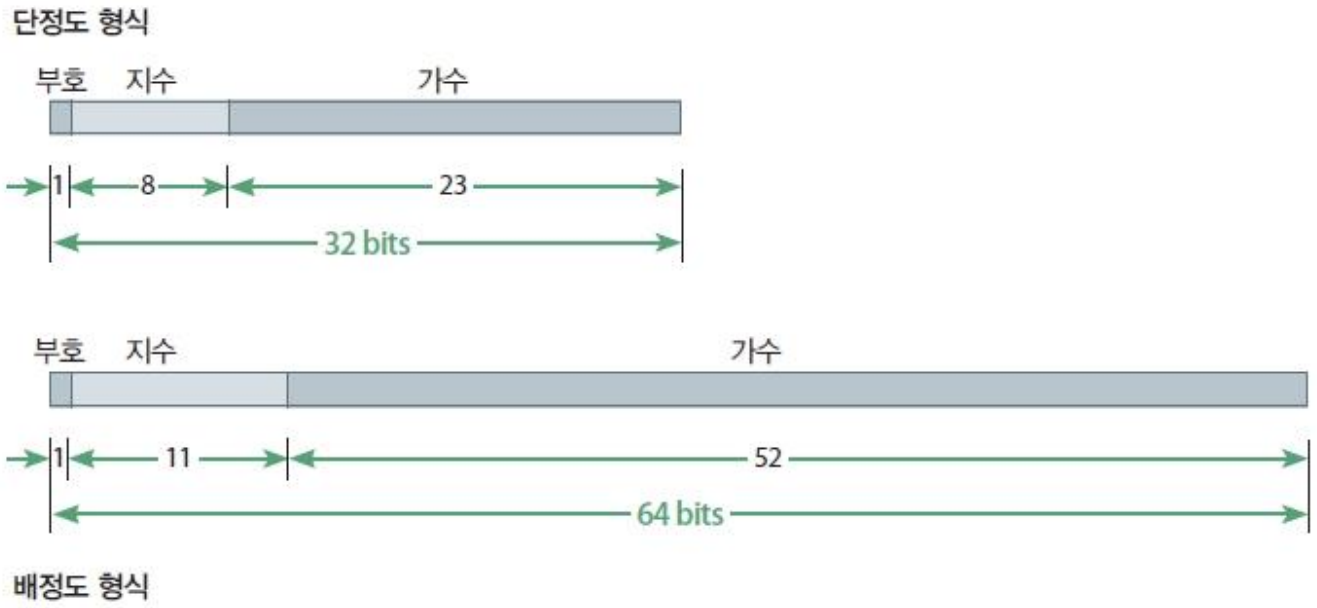


그림 3.19 ▸ IEEE 754 단정도 형식과 배정도 형식

지수를 표현하는 편향지수

단정도에서는 127 편향지수, 배정도에서는 1023 편향지수 사용

- **127 편향지수(127 biased exponent)**
  - 원래의 수에 127을 더하고, 그 수를 이진수로 표현한 수
- **1023 편향지수**
  - 원래의 수에 1023을 더한 수를 이진수로 표현한 수

표 3.10 지수부에 이용되는 127 편향지수와 1023 편향지수

원래의 수	127 편향지수		1023 편향지수	
	10진수	2진수(8비트)	10진수	2진수(11비트)
−1022	–		1	000 0000 0001
…	–		…	
−127	–		896	011 0000 0001
−126	1	0000 0001	897	011 0000 0010
…	…		…	
−2	125	0111 1101	1021	011 1111 1101
−1	126	0111 1110	1022	011 1111 1110
0	127	0111 1111	1023	011 1111 1111
1	128	1000 0000	1024	100 0000 0000
2	129	1000 0001	1025	100 0000 0001
…	…		…	
127	254	1111 1110	1150	100 0111 1110
128	–		1151	100 0111 1111
…	–		…	
1023	–		2046	111 1111 1110

부동소수점 수로의 변환

10진수 5.625가 메모리에 저장된 단정도 형식

- **5.625를 2진수로 변환하면 101.101**
  - 정규화
    - **부호는 양수이며 지수는 2이고 가수는 01101**

표 3.11 2진수 실수를 부동소수의 부호, 지수, 가수로 표현

원 실수	정규화	부호	지수	가수
101.101	$+1.01101 \times 2^2$	+	2	01101

- **부호가 양수: 부호부는 0**
- **지수부: 2 + 127인 129의 이진 표현 10000001**
- **가수부: 01101에 나머지 오른쪽 18개의 비트가 모두 0으로 채워진 이진 표현**

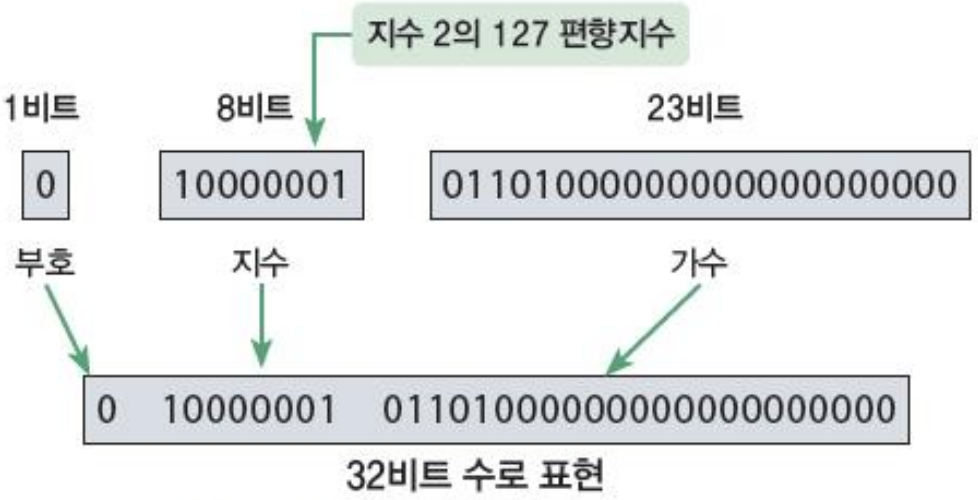


그림 3.20 ▸ 2진수 101.101의 부동소수 표현

부동소수의 저장 표현

10진수 0.15625를 2진수로 변환하면 0.00101이며, 이것을 정규화하면 1.01×2^{-3}

- **부호: 양수이므로 0**
- **지수: -3이므로 127 편향지수로 계산**
  - -3 + 127 = 124 = 01111100
- **가수**
  - 01이후에 나머지 21개의 비트를 모두 0으로 채움
    - **01000000000000000000000**

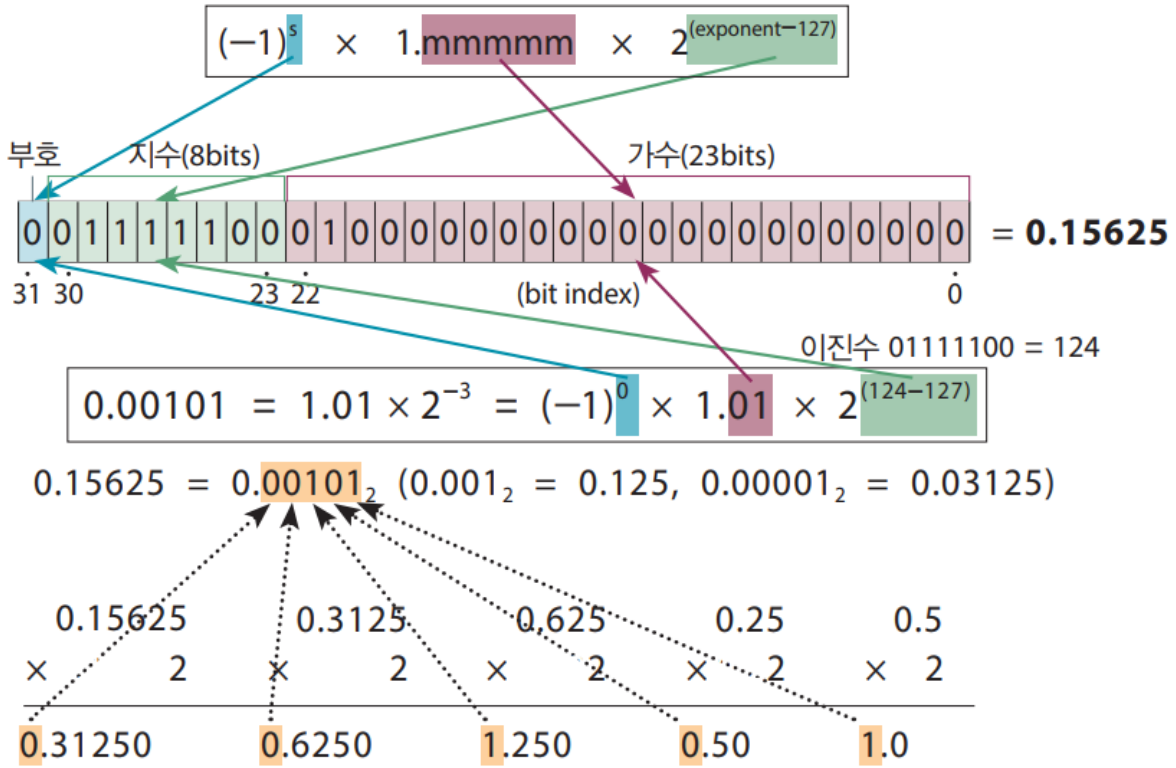


그림 3.23 ▸ 단정도 형식에서 10진수 0.15625의 2진수 변환

문자와 코드표

컴퓨터에서 문자는 하나의 정해진 수로 표현, 예로 문자 C는 1000011, O은 1001111로 표현

- **영문자는 7개의 비트의 조합으로 표현**
  - 일반적으로 n비트를 사용하면 총 2^n개의 서로 다른 문자 표현이 가능
  - 한데, 각각의 조합에 일정한 문자를 할당하여 지정한 것을 문자 코드(code)
- **국제 표준인 문자 코드**
  - 아스키코드
  - 유니코드

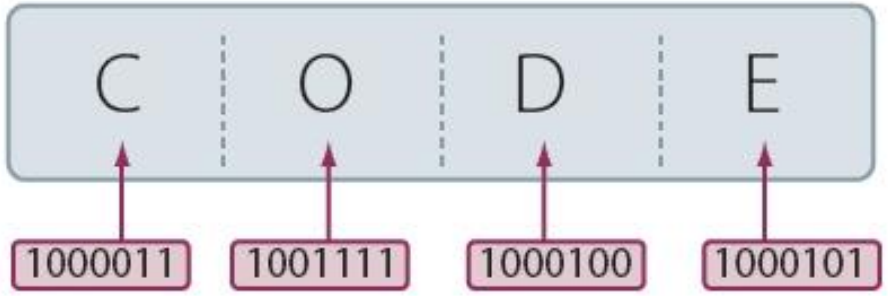


그림 3.24 ▸ 문자의 비트 표현 예

아스키 코드

미국 표준협회에서 국제적인 표준으로 정한 문자 코드 체계

- **ASCII**
  - American Standard Code for information Interchange
    - **7비트를 사용하여 128개의 문자, 숫자, 특수문자 코드를 규정**
  - 아스키코드가 7비트를 이용하지만 실제로 한 문자는 8비트인 1바이트에 저장
    - **한 문자에 시작은 0으로, 나머지는 코드 값으로 구성**

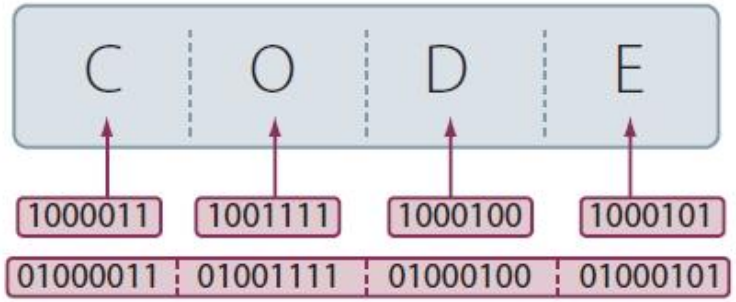


그림 3.26 ▸ "code" 문자열이 저장된 1바이트의 메모리 내부

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
000	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
001	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
010	Space	!	"	#	$	%	&	'	(	)	*	1	,	–	.	/
011	0	1	2	3	4	5	6	7	8	9	:	;	〈	5	〉	?
100	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
101	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
110	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
111	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

그림 3.25 ▸ 아스키코드

유니코드

세계의 문자를 모두 표현하기 위해 2바이트 즉, 16비트로 확장된 코드 체계가 유니코드

- **유니코드**
  - 전세계 모든 언어를 하나의 코드 체계 안으로 통합하기 위하여 만들어진 코드
  - 전 세계의 문자를 모두 표현하기 위한 2바이트 인 16비트로 확장된 코드 체계

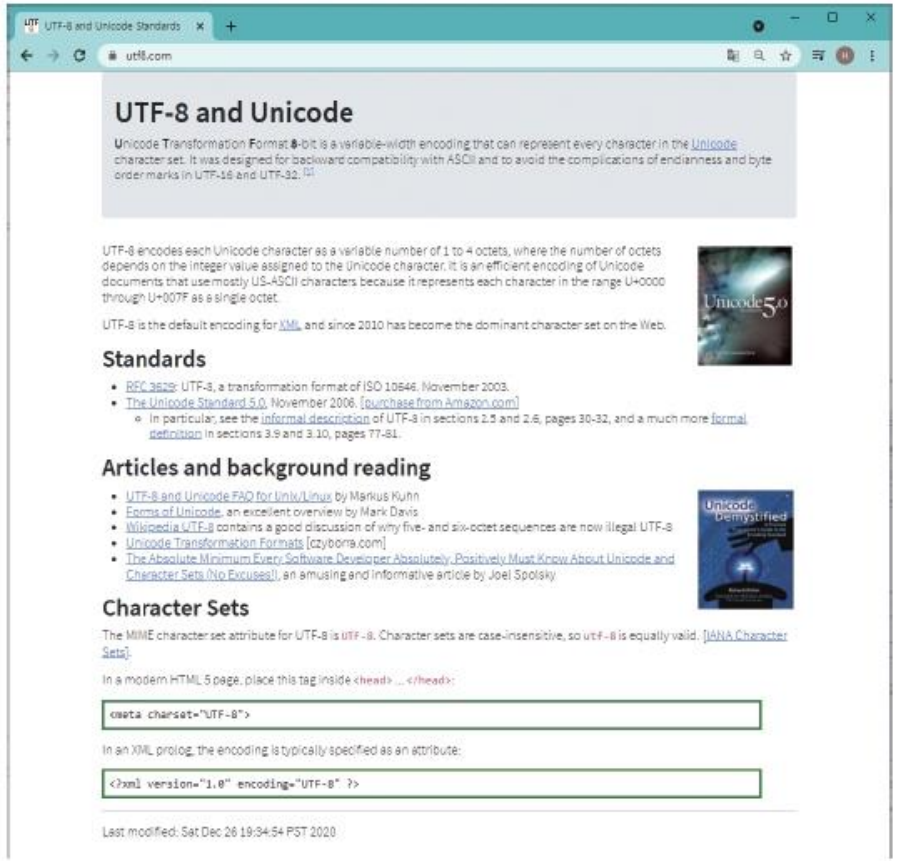


그림 3.28 ▸ UTF-8 자료 페이지

논리 표현

하나의 비트 정보도 0과 1이므로 이를 각각 거짓과 참으로 표현

- **두 가지 정보를 논리 값**
  - 참(true)과 거짓(false)
- **부울대수(Boolean Algebra)**
  - 이진 논리 변수와 AND, OR, NOT의 논리 연산을 이용한 수학
  - 논리 회로를 수학적으로 해석하기 위해 영국의 수학자 부울(George Bool)이 제창한 기호 논리학의 한 분야
  - 컴퓨터가 정보를 처리하는 방식에 대하여 이론적인 배경을 제공

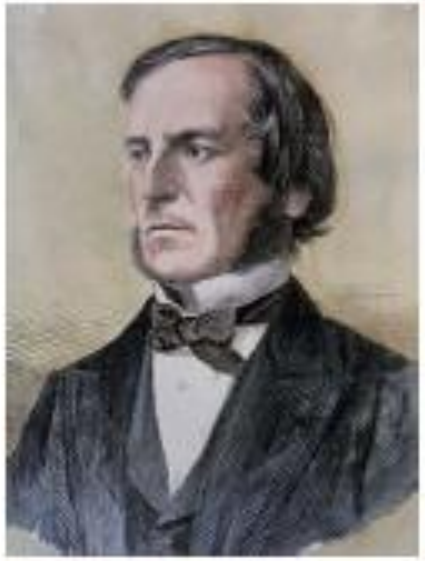


그림 3.29 ▸ 조지 부울

정수 연산

컴퓨터에서 정수의 덧셈과 뺄셈 연산 방법을 학습, 정수의 뺄셈 연산에서 2의 보수 표기를 이용

- **정수의 덧셈**
  - 정수의 연산에서 주의할 것은 정해진 비트를 넘어서는 올림은 무시
- **정수의 뺄셈**
  - 정수의 뺄셈에서는 2의 보수를 이용한 음수의 표현 방법을 이용
    - **a – b의 연산은 a + (– b)로 변환하여 덧셈을 수행**
  - 4 – 4
    - **4 + (-4)**

	17 :	0	0	0	1	0	0	0	1
	+ 23 :	0	0	0	1	0	1	1	1
결과	40 :	0	0	1	0	1	0	0	0

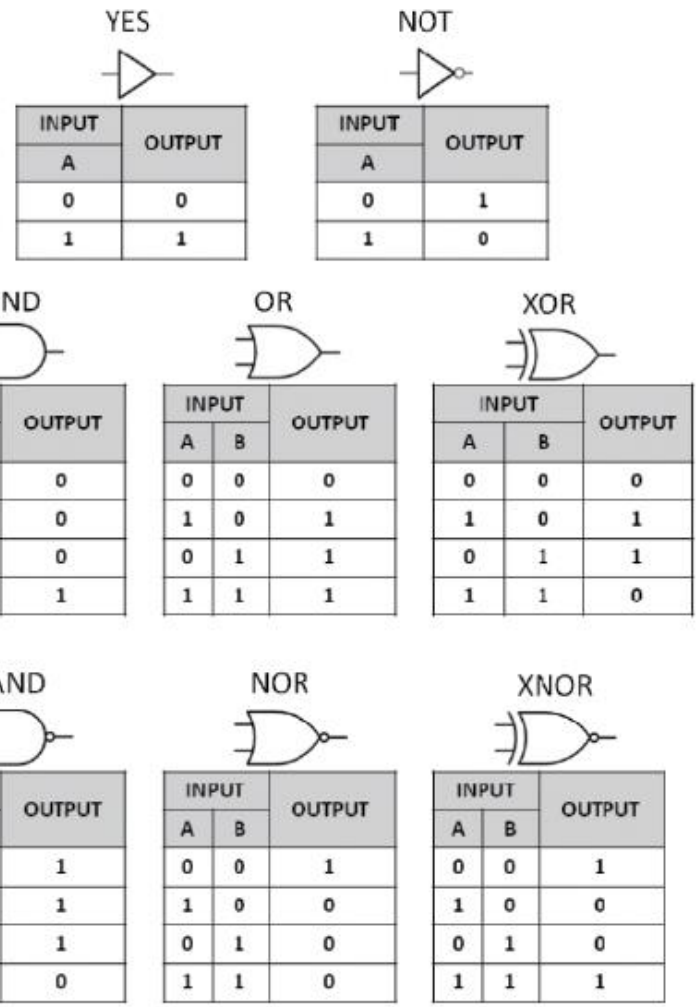
그림 3.30 ▸ 정수의 덧셈 연산

그림 3.31 ▸ 정수의 뺄셈 연산

논리 연산자

논리 연산 NOT, AND, OR

- **단항 연산자와 이항 연산자로 구분**
  - NOT
    - **단항 연산자는 NOT 연산자**
      - **하나의 항만이 연산에 참여**
  - AND, OR
    - **이항 연산자 AND, OR 연산자**
      - **두 개의 항이 연산에 참여**



INPUT A	OUTPUT
0	0
1	1

INPUT A	OUTPUT
0	1
1	0

INPUT A	INPUT B	OUTPUT
0	0	0
1	0	0
0	1	0
1	1	1

INPUT A	INPUT B	OUTPUT
0	0	0
1	0	1
0	1	1
1	1	1

INPUT A	INPUT B	OUTPUT
0	0	0
1	0	1
0	1	1
1	1	0

INPUT A	INPUT B	OUTPUT
0	0	1
1	0	1
0	1	1
1	1	0

INPUT A	INPUT B	OUTPUT
0	0	1
1	0	0
0	1	0
1	1	0

INPUT A	INPUT B	OUTPUT
0	0	1
1	0	0
0	1	0
1	1	1

그림 3.32 ▸ 논리연산자 AND, OR, XOR, NAND, NOR, XNOR의 게이트 표시와 진리표

논리 연산 법칙

논리 연산에서 자주 이용되는 연산의 법칙을 정리한 표

표 3.13 논리연산 법칙

① 항등 법칙	$0 + x = x,\ 1 + x = 1$	$0 \cdot x = 0,\ 1 \cdot x = x$
② 동일 법칙	$x + x = x$	$x \cdot x = x$
③ 보수 법칙	$x + x' = 1$	$x \cdot x' = 0$
④ 부정의 부정	$(x')' = x$	
⑤ 교환 법칙	$x \cdot y = y \cdot x$	$x + y = y + x$
⑥ 결합 법칙	$x + (y + z) = (x + y) + z$	$x \cdot (y \cdot z) = (x \cdot y) \cdot z$
⑦ 분배 법칙	$x \cdot (y + z) = x \cdot y + x \cdot z$	$x + y \cdot z = (x + y) \cdot (x + z)$
⑧ 드모르강 법칙	$(x + y)' = x' \cdot y'$	$(x \cdot y)' = x' + y'$
⑨ 흡수 법칙	$x + (x \cdot y) = x$	$x \cdot (x + y) = x$
흡수 법칙 증명	$x + (x \cdot y) = x \cdot 1 + (x \cdot y)$ $= x \cdot (1 + y)$ $= x \cdot 1$ $= x$	$x \cdot (x + y) = (x + 0) \cdot (x + y)$ $= x + (0 \cdot y)$ $= x + 0$ $= x$

논리회로 설계

논리연산자들은 게이트(gate)라는 물리적인 장치로 구현될 수 있으므로 필요한 논리회로를 설계

- **게이트**

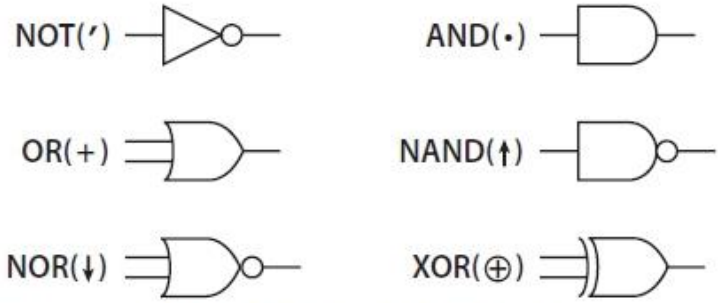


그림 3.34 ▸ 논리연산자의 기호 표현

- **논리회로 설계**

$X_2' \cdot (X_1' + X_1 \cdot X_3)$

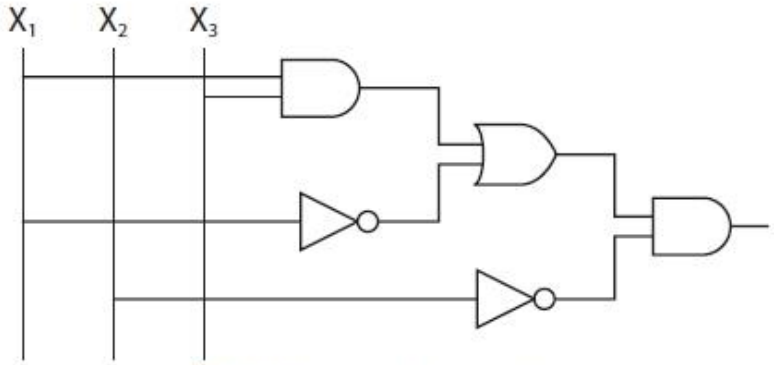


그림 3.35 ▸ 논리회로 표현 예